AI Capability Exercise

Participant Guide — Build & Grow Your AI Workflow

A hands-on exercise for data engineers in the competency to strengthen and show how you work with AI in realistic data engineering. You'll get a feedback report and a personalized growth path — this is for development, not a graded test.

Contents

1. What This Is
2. Who Takes Part
3. Time and Effort
4. What You Get Out of It
5. How the Exercise is Structured
    - Part A: AI Workflow Foundation
    - Part B: Medallion Architecture Data Pipeline Project
    - Part C: Submission and Reflection
6. Part A: AI Workflow Foundation — Details
7. Part B: Medallion Architecture Data Pipeline Project — Details
8. Required Repository Structure
9. Submission Templates
10. Tool-Specific Expectations (Cursor)
11. What Counts as Complete
12. How to Take Part
13. What Good Looks Like
14. Your Growth Path
15. Summary

1. What This Is

This is a hands-on capability exercise to help you develop — and make visible — how you use AI tools effectively, responsibly, and practically across the data engineering lifecycle. Everyone in the competency takes part; it is a shared part of how we build AI capability, not something a few people are singled out for.

It is not a graded test. You will build a complete Databricks medallion data pipeline (Bronze → Silver → Gold → Dashboard) and show your thinking across sample data generation, ingestion, data quality

validation, aggregations, and visualization. In return you get a feedback report and a clear sense of what to grow next.

What matters is not only whether the final pipeline works, but **how you used AI** for design, implementation, validation, testing, debugging, and reflection. Making your thinking visible is the point.

2. Who Takes Part

All data engineers in the competency, from individual contributor to Tech Lead level, working across cloud platforms and data stack tools (Databricks, PySpark, SQL, Python). Because everyone does it, there is a shared baseline and no one is measured against a different bar than their peers.

3. Time and Effort

The exercise is self-paced and meant to be completed within **three weeks**. You may work in any order; there is no required day-wise plan. Share your work by the agreed date.

Expected effort: The mandatory Core of the project is scoped for roughly **20-25 focused hours**. The rest of the time goes into the lifecycle artifacts — requirement analysis, prompt history, data quality validation notes, testing and debugging notes, reflection — which are the main things the feedback looks at.

Do not expand the pipeline complexity at the expense of these artifacts.

4. What You Get Out of It

You receive a feedback report: your strengths, your growth areas, and concrete next steps for developing your AI-assisted data engineering workflow. The report also gives a sense of where you currently sit in the AI capability framework and what would move you forward — think of it as a snapshot and a direction, not a grade.

Feedback focuses on areas like:

- Requirement analysis and problem understanding
- Prompting and context-setting with AI tools
- Tool workflow and integration
- Data pipeline design (Bronze/Silver/Gold thinking)
- Code quality and maintainability
- Data quality validation depth
- Testing and validation approach
- Debugging methodology

- Data contracts and schema thinking
- Documentation and ownership
- Responsible AI judgment

5. How the Exercise is Structured

It has three parts (the percentages show where to put your effort):

Part	Focus	Emphasis
Part A	AI Workflow Foundation	20%
Part B	Medallion Architecture Data Pipeline Project (Core + optional Stretch)	60%
Part C	Submission and Reflection	20%

6. Part A: AI Workflow Foundation — Details

Objective

Show that you understand how AI should be used in practical data engineering — thoughtfully, not as a simple code-generation shortcut.

Expected Submission

Submit a document named **tool-workflow.md** covering:

- Primary AI tool used (Cursor, Claude, etc.)
- How you provide project context to the tool
- How you use AI for requirement analysis
- How you use AI for pipeline design (Bronze/Silver/Gold - Medallion Architecture)
- How you use AI for code generation (Python/PySpark/SQL)
- How you validate AI-generated code and logic
- How you use AI for testing and validation
- How you use AI for debugging (issues, root causes)
- How you use AI for data quality checks
- What information you avoid sharing unnecessarily with AI tools (e.g., real customer PII)
- How you would reuse this workflow in a real production pipeline
- Lessons learned: what worked, what didn't

7. Part B: Medallion Architecture Data Pipeline Project — Details

Objective

Demonstrate practical AI-assisted delivery through a realistic Medallion Architecture data engineering assignment.

Business Context

Problem Statement:

An e-commerce company ingests daily sales data from multiple sources (customer database, order system, product catalog) into Databricks. They need to:

- **Bronze Layer:** Ingest raw CSV files from S3/DBFS
- **Silver Layer:** Apply data quality checks, clean and validate data
- **Gold Layer:** Create business-ready aggregations for analytics
- **Dashboard:** BI dashboards for business stakeholders

Common Technical Requirements

Whichever option you choose, every submission must include:

- ✅ Sample data generator script (creates realistic CSV files with intentional quality issues)
- ✅ Bronze layer ingestion code (Python/PySpark)
- ✅ Silver layer validation code (all 4 quality checks working)
- ✅ Gold layer aggregation code (all 4 aggregations)
- ✅ Dashboard queries (3+ SQL queries for visualizations)
- ✅ Database schema or setup script
- ✅ Seed/sample data (customers, orders, products CSVs)
- ✅ Input validation and error handling
- ✅ Data quality reporting
- ✅ At least one meaningful test tier (data quality tests, pipeline tests)
- ✅ README setup instructions
- ✅ **Full prompt history** (CRITICAL)
- ✅ All planning, design, testing, debugging, and reflection artifacts in the repository

The full set of lifecycle artifacts is required regardless of stretch tier. Only the application scope is small — the artifacts are the point.

Data Schema & Technical Setup

Source Files (S3/DBFS CSVs)

Table 1: customers.csv

- customer_id (INT, Primary Key)
- customer_name (STRING)
- email (STRING)
- country (STRING)
- signup_date (DATE)
- customer_segment (STRING: Premium/Standard/Basic)
- lifetime_value (DECIMAL)

Sample rows: 10,000 customers | File size: ~500 KB

Table 2: orders.csv

- order_id (INT, Primary Key)
- customer_id (INT, Foreign Key → customers)
- order_date (DATE)
- product_id (INT, Foreign Key → products)
- quantity (INT)
- unit_price (DECIMAL)
- total_amount (DECIMAL)
- order_status (STRING: Pending/Completed/Cancelled)
- payment_date (DATE, nullable)

Sample rows: 100,000 orders | File size: ~2-3 MB

Table 3: products.csv

- product_id (INT, Primary Key)
- product_name (STRING)
- category (STRING)
- price (DECIMAL)
- cost (DECIMAL)
- stock_quantity (INT)
- reorder_level (INT)

Sample rows: 500 products | File size: ~50 KB

Data Quality Issues (Intentional)

Your sample data should include realistic quality issues for the Silver layer to catch:

customers.csv:

- 50 rows with NULL email (completeness)
- 10 rows with duplicate customer_id (uniqueness)

orders.csv:

- 100 rows with NULL customer_id (completeness)
- 200 rows with NULL product_id (completeness)
- 50 rows with customer_id not in customers table (referential integrity)
- 30 rows with product_id not in products table (referential integrity)
- 20 duplicate order_id rows (uniqueness)

Total issues: ~700 problematic rows out of ~100,000 (0.7% — realistic data quality)

Core Logic for Bronze to Gold layer

Five main components:

1. Sample Data Generation

- Use Cursor to help design a Python/PySpark script
- Generate all three CSVs with realistic data
- Intentionally introduce the quality issues listed above
- Document how data was generated and why quality issues exist

2. Bronze Layer — Ingestion

- Read CSVs from S3/DBFS into Databricks
- Create Bronze tables (raw, unchanged data)
- Handle schema inference and data types
- Log ingestion metadata (row counts, timestamp)
- No transformations or cleaning — just raw ingest

3. Silver Layer — Data Quality & Validation

- Implement below quality checks:
    - **Completeness:** No NULLs in critical fields (email, customer_id, product_id)
    - **Uniqueness:** No duplicate rows (order_id, customer_id)
    - **Referential Integrity:** Foreign keys exist (every customer_id, every product_id)
- Flag bad rows (don't delete — add quality_check_result column)
- Generate quality metrics report (% passed each check)

4. Gold Layer — Aggregations & Analytics

Build three aggregation tables:

A) Sales by Product:

- product_id, product_name, category
- total_orders, total_revenue, avg_order_value

B) Revenue by Customer:

- customer_id, customer_name, customer_segment
- total_orders, total_revenue, avg_order_value, lifetime_value_actual

C) Customer Segmentation:

- segment_type (High-Value/Repeat/One-Time/Inactive)
- customer_count, avg_revenue, total_revenue

5. BI Dashboard

- Create Databricks SQL Dashboard with 3+ tiles
- Visualizations: Top 10 products by revenue (bar), Customer revenue distribution (histogram), Customer segmentation (pie)
- Write queries, configure visualizations, add filters

Core Acceptance Criteria:

- [ ] Sample data generated (3 CSVs with intentional issues)
- [ ] Bronze layer ingests all three sources successfully
- [ ] Silver layer implements all four quality checks
- [ ] Quality report shows % passed for each check
- [ ] Gold layer produces all three aggregation tables
- [ ] Aggregation calculations are correct (sum, count, avg, etc.)
- [ ] Dashboard displays all 3+ visualizations
- [ ] All code is readable, commented, documented
- [ ] README setup instructions work end-to-end
- [ ] Data quality tests pass (verify checks catch intentional issues)

8. Required Repository Structure

Submit a Git repository following this structure as closely as possible:

None

```
databricks-medallion-pipeline/
├── README.md
├── candidate-info.md
├── tool-workflow.md                    # Part A: AI Workflow Foundation
├── requirements-analysis.md
├── design-notes.md
├── data-model.md
├── data-quality-strategy.md
│
├── src/
│   ├── data_generation/
│   │   ├── generate_sample_data.py
│   │   └── DATA_GENERATION_NOTES.md
│   ├── bronze/
│   │   ├── 01_ingest_customers.py
│   │   ├── 02_ingest_orders.py
│   │   ├── 03_ingest_products.py
│   │   └── ingest_all.py
│   ├── silver/
│   │   ├── 01_quality_completeness.py
│   │   ├── 02_quality_uniqueness.py
│   │   ├── 03_quality_type_validation.py
│   │   ├── 04_quality_referential_integrity.py
│   │   ├── 05_quality_business_logic.py
│   │   └── create_silver_tables.py
│   ├── gold/
│   │   ├── 01_sales_by_product.sql
│   │   ├── 02_revenue_by_customer.sql
│   │   ├── 03_daily_weekly_trends.sql
│   │   ├── 04_customer_segmentation.sql
│   │   └── create_gold_tables.py
│   └── dashboard/
│       ├── dashboard_queries.sql
│       └── DASHBOARD_GUIDE.md
│
├── data/
│   ├── customers.csv
│   ├── orders.csv
│   └── products.csv
│
├── database/
│   ├── schema.sql
│   ├── seed-data-notes.md
│   └── setup-notes.md
│
```

```
├── debugging-notes.md
├── reflection.md
├── final-ai-usage-summary.md
│
└── ai-prompts/
    ├── data-generation.md
    ├── bronze-layer.md
    ├── silver-layer.md
    ├── gold-layer.md
    ├── dashboard.md
    ├── debugging.md
    └── documentation.md
```

---

9. Submission Templates

Use these as starting structures for required artifacts — they are a floor, not a limit.

Candidate Information

File: candidate-info.md

```
None
# Candidate Information

**Name:** [Your Name]
**Role:** [SE / SSE / ATL/TL / other]
**Primary Technology Stack:** Python / PySpark, SQL, Databricks
**Primary AI Tool Used:** Cursor / Claude / other
**Project Option Selected:** Data Pipeline (Medallion Architecture)
**Assessment Start Date:** [Date]
**Submission Date:** [Date]

## Tools & Environment
- Databricks: Community Edition / other
- Languages: Python, PySpark, SQL
- Libraries: PySpark, Delta Lake, pandas
- AI Tool: [Cursor / Claude]

## Setup Summary
[Quick reference for how to run the pipeline — expanded in README.md]
```

Requirement Analysis

File: requirements-analysis.md

```
None
# Requirement Analysis

## Problem Statement
[Your understanding of the e-commerce sales pipeline problem in your own words]

## Functional Requirements
- [requirements]

## Non-Functional Requirements
- [non functional requirements]

## Assumptions
- [assumptions]

## Edge Cases
- [edge cases]

## Clarifications Needed
- [carifications]
```

Design Notes

File: design-notes.md

```
None
# Design Notes

## Architecture Overview
- [High-level design of Bronze → Silver → Gold → Dashboard]
## Data Model & Schema
- [Descriptions of customers, orders, products tables]
## Bronze Layer Design
- [bronze layer]
## Silver Layer Design
- [silver layer]
## Gold Layer Design
- [gold layer]
## Data Quality Validation Strategy
- [data quality]
## Debugging Approach
- [debugging]
```

Data Quality Strategy

File: data-quality-strategy.md

```
None
# Data Quality Strategy

## Quality Checks Overview

### 1. Completeness Check
- **What:** No NULLs in critical fields
- **How:** COUNT NULL values in email, customer_id, product_id
- **Threshold:** >99% complete
- **Result:** Flag rows with NULLs

### 2. Uniqueness Check
- **What:** No duplicate rows
- **How:** Check for duplicate order_id, customer_id
- **Threshold:** 100% unique
- **Result:** Flag duplicate rows

### 3. Referential Integrity
- **What:** Foreign keys exist in parent tables
- **How:** Check customer_id in customers, product_id in products
- **Threshold:** >99.9% valid
- **Result:** Flag orphan records

## Quality Metrics Report
[How you'll present % passed per check]

## Sample Data Quality Issues
[List the ~700 intentional issues in your sample data]
```

AI Prompts — Organized by Activity

Files: ai-prompts/{activity}.md

For each activity, capture prompt history showing:

- **Prompt text** (or summary)
- **AI response** (summary or key excerpt)
- **What you accepted** (and why)
- **What you changed** (and why)
- **What you rejected** (and why)

Example: ai-prompts/data-generation.md

```
None
# AI Prompts – Data Generation

## Prompt 1: Initial Data Generation Script

**PROMPT SENT:**
"Generate Python script to create realistic e-commerce customer data.
I need 10,000 rows with these fields: customer_id (INT), customer_name (STRING),
email (STRING), country (STRING), signup_date (DATE between 2020-2024),
customer_segment (Premium/Standard/Basic), lifetime_value (DECIMAL).
Include realistic values like actual names, valid email formats, and random
dates."

**AI RESPONSE SUMMARY:**
[Cursor generated Python script using faker library to create realistic data]

**YOUR EVALUATION:**
✓ **What was good:**
- Used faker for realistic names and emails
- Date range correct (2020-2026)
- Customer segments randomized properly

✗ **What needed fixing:**
- Some customers had signup_date in future
- No intentional quality issues (needed 50 NULL emails, 10 duplicates, etc.)
- Missing lifetime_value calculations

△ **Missing:**
- No NULL values as needed for quality testing

## Iteration 1: Adding Quality Issues

**PROMPT SENT:**
"Modify the script to introduce intentional quality issues for testing:
- 50 rows with NULL email
- 10 rows with duplicate customer_id
- 30 rows with signup_date > today()
Keep the rest realistic. Add comments explaining the quality issues."

**AI RESPONSE SUMMARY:**
[Cursor modified script to add quality issues and comments]

**YOUR EVALUATION:**
✓ **ACCEPTED** - Modifications correct, quality issues intentional and commented
```

```
**FINAL DECISION:** Use this version as `generate_sample_data.py`

---

## Prompt 2: Order Data Generation

**PROMPT SENT:**
"Generate Python script for 100,000 realistic e-commerce order rows...
[similar structure for orders]"

**[Continue pattern for each major prompt]**
```

Reflection

File: reflection.md

```
None
# Reflection
## What I Built
-
## How I Used AI (Across the Lifecycle)
-
## What AI Helped With Most
-
## What AI Got Wrong
-
## How I Validated AI Output
-
## What I Would Improve Next
-
## Reusable Workflow
```

10. Tool-Specific Expectations: Cursor

Expectations for Cursor Users

Submit tool-specific/cursor-workflow/ with:

- **project-context.md** — How you set up project context for Cursor
- **spec.md** — Your design/specification document
- **cursor-rules-or-instructions.md** — Cursor rules, .cursorrules file, or instructions you used

- **task-breakdown.md** — Tasks as you defined them to Cursor

Show Evidence Of:

✅ **Persistent project context** — How you provided context to Cursor repeatedly
✅ **Iteration** — Multiple refinement cycles; accepting some suggestions, rejecting others
✅ **Validation** — How you verified Cursor-generated code worked before accepting it

Strong Cursor Usage Shows:

- You wrote a design spec, shared it with Cursor, and built against it
- You used .cursorrules or similar to enforce project standards
- Your commit history shows you iterating: accepting → testing → fixing → refining
- Your prompts were specific ("Generate quality check for completeness on these 3 fields") not vague ("write data quality code")
- You tested Cursor-generated SQL/Python/PySpark before deploying it
- You rejected suggestions that didn't match your architecture

Weak Cursor Usage Shows:

- One-line prompts only; no context provided upfront
- Copying code directly without understanding it
- No evidence of testing or validation
- Prompts like "generate code" with no specification
- Missing git history or shallow commits
- No documented reasoning for accepting/rejecting suggestions

---

11. What Counts as Complete

For your feedback to be useful, your submission should include:

✅ A working end-to-end pipeline (Bronze → Silver → Gold → Dashboard)
✅ Sample data generator with realistic quality issues
✅ All four quality checks implemented and working
✅ All three Gold layer aggregations
✅ Dashboard with 3+ SQL queries and visualizations
✅ Database schema/setup script and seed data
✅ README with working setup instructions
✅ Basic test suite (data quality tests, pipeline integration tests)
✅ Full prompt history with all AI interactions documented

✅ Requirement analysis, design notes, test strategy
✅ Debugging notes and code review notes
✅ Reflection on what you learned

If pieces are missing you will still get feedback — it just won't reflect the full picture, and the growth pointers will focus on filling those gaps.

12. How to Take Part

Once your project is ready, you share it through a single online form. There is no review call to book and nothing to host or deploy.

What You Share

- A link to your Git repository using ttn email id. The same repo can be cloned to the databricks community edition (free) for development/testing/validations.
- Short written answers to questions in the form about your work

About the Questions

The questions ask you to explain your work in your own words:

- Your understanding of the medallion architecture problem
- How you used AI across data generation, ingestion, validation, aggregation
- Key design and implementation decisions made through AI
- Your testing and validation approach
- How you validated AI output
- What you'd improve next

Be specific and honest. This is for your own development, so there is no value in generic or inflated answers — they just produce less useful feedback. The clearer picture you give of how you actually worked, the more useful your growth pointers will be.

Follow-Up and Coaching

Your work is reviewed and a feedback report is shared with you. Sometimes a mentor or your competency owner may follow up to talk through your project and coach you on next steps — a short, supportive conversation, not a re-examination. This is part of how the competency helps you grow.

13. What Good Looks Like

The feedback weighs how you used AI across the lifecycle and how well you understand and own your solution — not just whether the pipeline runs.

Strong Work Usually Shows

✅ **Clear requirement understanding** — Good breakdown of Bronze/Silver/Gold layers, acceptance criteria
✅ **Well-documented AI prompts** — Context-setting, refinement, correction of wrong suggestions
✅ **Working pipeline end-to-end** — All layers function, data persists, dashboard displays correctly
✅ **Data quality thinking** — All checks work, quality report is clear, intentional issues caught
✅ **Clean, maintainable code** — Readable, documented, follows naming conventions
✅ **Meaningful testing** — Quality validation tests, integration tests, evidence of debugging
✅ **Honest reflection** — Can explain trade-offs, show what you learned, reusable patterns documented

Weaker Work Usually Shows

❌ **Direct copy-paste from AI** — Little understanding; missing requirement analysis
❌ **Shallow prompt history** — No clear design; prompts lack context
❌ **Broken setup instructions** — README doesn't work; no data persistence
❌ **Superficial testing** — No evidence quality checks catch their issues
❌ **Generic documentation** — Can't explain the code; no ownership
❌ **Missing artifacts** — Prompt history, reflection, testing, or debugging notes absent

In short: The journey and your understanding matter as much as the final code. Make your thinking visible.

14. Your Growth Path

Whatever your starting point, the feedback comes with a direction for what to develop next:

Building the basics:

- Focus on requirement analysis and providing context to your AI tool
- Get to a working, tested pipeline end-to-end
- Next step: Move toward solid practice

Solid and growing:

- You produce working AI-assisted pipelines
- Quality checks and aggregations work reliably
- Next step: Deepen testing, review quality, and debugging maturity

Strong across the lifecycle:

- You use AI well end-to-end (design → implementation → testing)
- Can explain your decisions and trade-offs
- Next step: Advanced testing, performance optimization, reusable patterns/libraries

Ready to lead:

- Mature workflow with reusable prompt templates, specs, or pipeline frameworks
- Can mentor others on AI-assisted data engineering
- Natural point to share practices and contribute to org standards

Your competency owner uses the report and this direction within your normal growth conversations — it informs where you are in the AI capability framework rather than stamping a grade on you.

15. Summary

You'll complete a self-paced, three-week AI-assisted data engineering exercise using Cursor (or approved AI tool).

You will:

- Build a complete Databricks medallion pipeline (Bronze → Silver → Gold → Dashboard) for e-commerce sales data
- Create realistic sample data with intentional quality issues (700 problematic rows)
- Implement four data quality checks in the Silver layer
- Create three aggregations in the Gold layer
- Build a dashboard with 3+ visualizations
- Show your thinking: requirement analysis, design, prompt history, testing, debugging, reflection

What you'll submit:

- Git repository with working code, setup instructions, database scripts, seed data
- AI prompt history (organized by activity)
- Requirement analysis, design notes, test strategy
- Debugging and code review notes
- Reflection on what you learned

What you'll get back:

- Feedback report on your strengths, growth areas, and next steps
- Clear sense of where you sit in the AI capability framework
- Possibly a short coaching conversation with your competency owner

Everyone in the competency does this — it is a shared part of building how we all work with AI. You are not graded; you are developed.